



Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

Names: Student 1 - Sean Sweeney (22408204)
Student 2 - Shane Mahon (22376743)

Project Title: Zeph

Document: Functional Specification

Table of Contents

1. Introduction.....	2
1.1 Overview.....	2
1.2 Business Context	2
1.3 Glossary.....	3
2. General Description.....	4
2.1 Product / System Functions	4
2.2 User Characteristics and Objectives.....	4
2.3 Operational Scenarios.....	5
2.4 Constraints.....	8
3. Functional Requirements.....	10
3.1 - FR01: Flight Registration.....	10
3.2 - FR02: Mock Oracle Service.....	10
3.3 - FR03: Claim Eligibility Calculation.....	11
3.4 - FR04: Claim Approval / Rejection.....	11
3.5 - FR05: Claim Evidence Collection.....	12
3.6 - FR06: Live Claim Status.....	12
4. System Architecture.....	13
5. High-Level Design.....	14
6. Preliminary Schedule.....	18
6.1 Gantt Chart.....	20
7. Appendices.....	21

1. Introduction

1.1 Overview

Zeph is a web application designed to automate the process of handling flight delay compensation claims under EU Regulation EU261/2004¹. This regulation states that if a passenger arrives at their destination three or more hours late they are entitled to compensation, unless the delay resulted from extraordinary circumstances outside the airline's control. Currently, if a passenger wants to make a claim under this regulation, passengers must manually submit claims through complex, slow and intentionally obstructive airline processes. Zeph addresses these issues by using blockchain and smart contract technology to evaluate compensation eligibility and enforce response timeframes. This will make the whole process of making these claims much faster, easier and fairer for passengers.

1.2 Business Context

The app is a scenario of the commercial aviation and travel industry in the EU. According to the EU regulation EU/261, airlines are required by law to make amends to a passenger who experiences a significant delay (Three hours or more). Nevertheless, the process of claiming is so old-fashioned and it varies from one airline to another and usually takes a lot of time. Zeph eliminates this problem by creating a compensation system that is open and automated using blockchain technology. Through automating the process of determining the eligibility of the claim, the system encourages the law to be abided by and the rights of passengers to be respected by the airlines.

Operationally, Zeph can be a tool used by the Regulatory Authority to monitor the compliance of the regulation or as a service when systems of the airline are integrated. Also, Zeph may serve as a record of decisions on compensation that can be verified by regulators thus they can cut down on the time for investigations and administrative costs. Furthermore, Zeph fits well in the industry context with the EU emphasis on digital transformation, data transparency, and consumer protection². By being the technology that guarantees accountability, Zeph acts as a demonstration for the possible ways in which future policies could be implemented.

¹ EU/261/2004

- https://europa.eu/voeuropa/citizens/travel/passenger-rights/air/index_en.htm

² EU Data Act 2005

- <https://digital-strategy.ec.europa.eu/en/policies/data-act#:~:text=The%20Regulation%20on%20harmonised%20rules,application%20on%2012%20September%202025.>

1.3 Glossary

Feature/Component	Description
EU261 / EU261/2004	European regulation establishing passenger compensation rights for delayed/cancelled flights.
Smart Contract	Self-executing blockchain program enforcing claim evaluation logic.
Oracle	Backend service for submitting flight arrival data to the blockchain (mock, not real-time).
On-Chain	Data stored on the blockchain.
Off-Chain	Data stored in external databases for privacy (e.g., passenger personal information).
Claim	A structured record representing a passenger's compensation eligibility.
Ethereum	A decentralised blockchain network used to deploy and execute smart contracts
Supabase	A backend-as-a-service platform providing authentication, Postgres database storage and file storage used for managing users, claims and evidence
MetaMask	A browser-based digital wallet that allows users to connect to blockchain applications and sign transactions required for interacting with smart contracts.
EIP-1559	Ethereum's transaction fee mechanism that introduces a base fee and optional priority fee.
Ethereum Sepolia	Public Ethereum testnet used for development and testing. Used in Zeph so transactions can be made without real cost.

2. General Description

2.1 Product / System Functions

Passenger users may register flights and handle claims with minimal manual interaction thanks to the system's automated, blockchain-based architecture. Blockchain smart contracts manage claim eligibility, guaranteeing transparency and trust between the client and the airline. When certain criteria are satisfied, the system notifies the airline by changing the status of claims and then informing airlines of pending claims; this is done using a mocked oracle that will submit aircraft arrival times. Through a web-based interface, users may examine active claims, connect their digital wallet, and keep track of claim statuses. Because every transaction is safely documented on the blockchain, there is a tamper-proof audit trail that lowers the possibility of fraud and administrative mistakes.

2.2 User Characteristics and Objectives

The system targets travelers who want to manage their flight delay refund in a fast and clear manner. It is expected that the users will have a basic knowledge of web applications but no more than that with respect to blockchain technology. Therefore, customers will be able to send requests, check their claim, and receive payment without having any technical knowledge due to the interface's ease and intuitiveness.

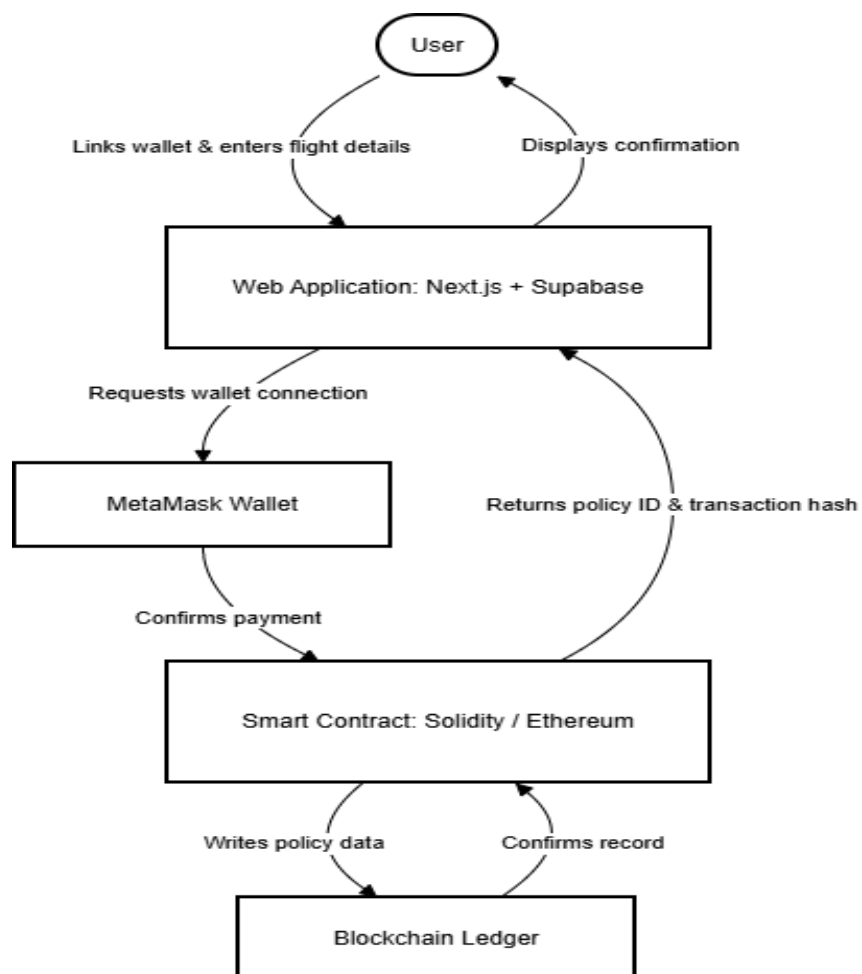
From the user's perspective, the primary goals are convenience, trust, and automation. They wish, among other things, to get real-time updates on their claims, monitoring done transparently, and being able to receive the money soon after a flight delay is confirmed. Even if it might not be feasible to have a complete connection with the real airline systems, the use of simulated travel data will make it possible to demonstrate automated claim verification and the sending of money through blockchain smart contracts.

From the airline's point of view, the system offers a simplified and effective method for handling and verifying compensation requests. The airline can check the claim details, including flight information, delay data, and supporting documentation, as soon as a user files a request via the platform. After the airline verifies the validity of the claim, the decision is safely recorded and carried out via the smart contract workflow. This strategy lowers administrative costs, guarantees clear auditability of choices, and assists airlines in adhering to compensation laws while enhancing customer satisfaction and trust.

2.3 Operational Scenarios

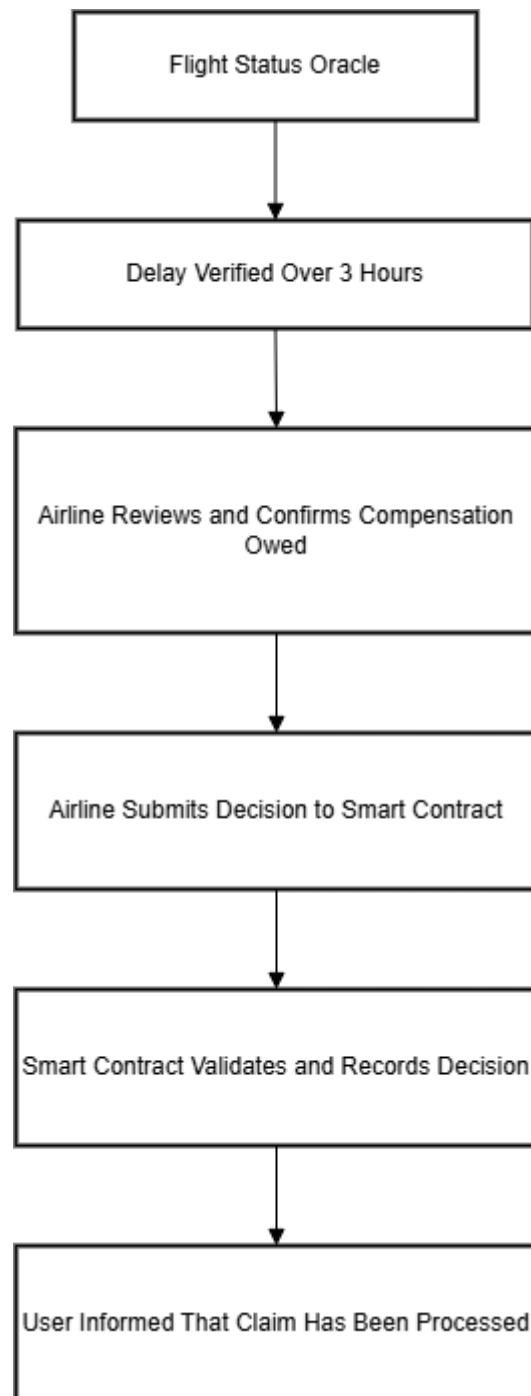
Scenario 1: Registering a claim

The user links their digital wallet to the web application. They register a flight by entering information about their next flight, such as the flight number, date, and boarding pass information. The system generates a new claim on the blockchain after the flight information has been verified. The details are permanently immutably in the blockchain ledger, and the user receives an on-screen confirmation.



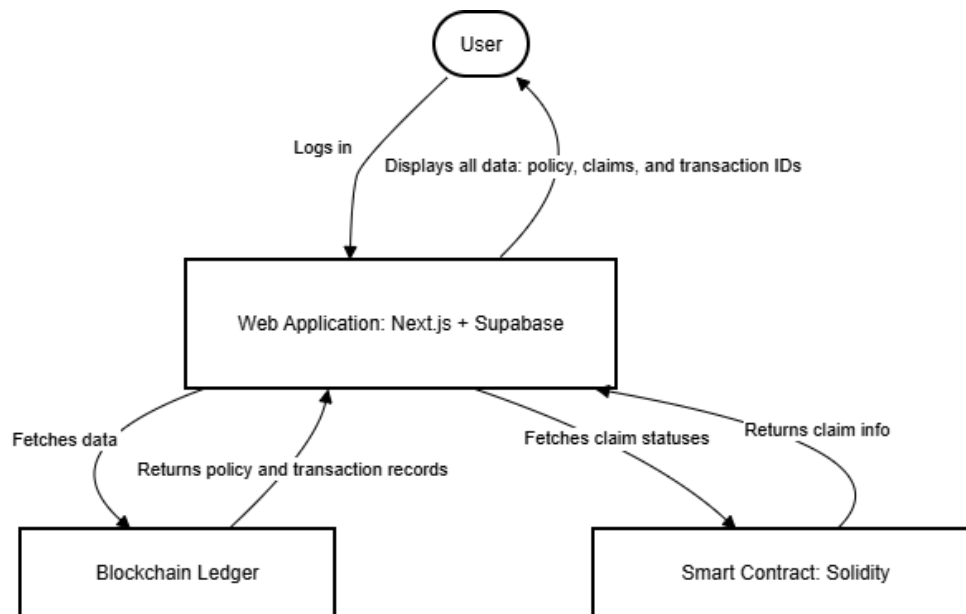
Scenario 2: Flight delay and payout

The system (or a simulated oracle) determines that the user's flight has been delayed more than the covered threshold on the day of departure (e.g., $\geq$ three hours). The smart contract will calculate the claim eligibility and then update the claims status to a pending state, after this the airline is notified and given a seven day period to initiate a payout or reject the claim and provide evidence.



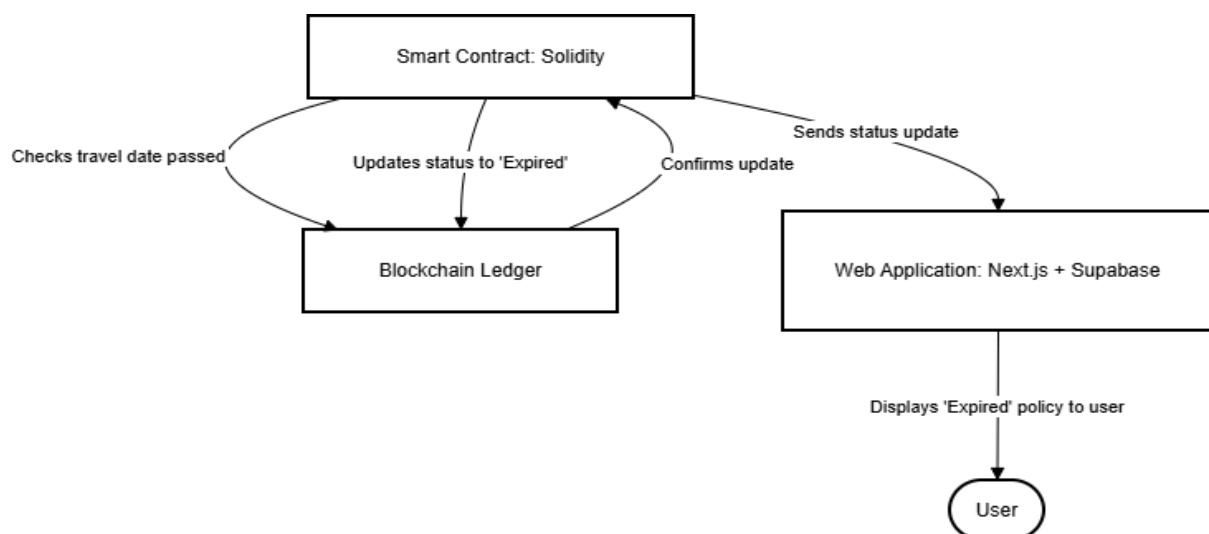
Scenario 3: Viewing claim history

To examine their current and previous claims, a user logs into the system. For verification, they can view their full claim history information. It is evident from the interface what state the claim is currently in (pending, approved etc). The user is reassured that their claims and payouts are safely documented thanks to this transparency.



Scenario 4: Claim expiration

The claim automatically expires after the travel date if the user's trip ends without any delays. Once the smart contract receives the arrival timestamp from the mocked oracle, if it deems that the plane arrived before the three hour delay threshold it will automatically change the status of the claim to expired. The user can still read the record on the blockchain for future reference after the smart contract changes the claim status to "expired."



2.4 Constraints

1. Limitations on Speed and Performance

- The Ethereum block confirmation speed, which is normally 12 to 15 seconds per block, limits the execution time of smart contracts. Finality cannot be assured right away.
- For typical interactions that don't include blockchain writes, user-facing answers from the web application should render in less than 200 milliseconds.
- Depending on network congestion, blockchain write operations (policy generation, payouts) might take anywhere from 10 to 60 seconds.
- Depending on the data source and implementation, Oracle updates must provide flight status updates within ± 5 minutes.

2. Web3 and Blockchain Restrictions

- EVM restrictions must be adhered to by smart contracts, including:
 - Gas limits
 - Storage expenses
 - Deterministic requirements for execution
- Every policy and claim logic must be implemented on an EVM-compatible chain and written in Solidity.
- A Web3 wallet (MetaMask) and enough money to pay for gas, unless subsidized, are prerequisites for interacting with the blockchain.
- EIP-1559 fee mechanics for transactions must be supported by the system.

3. Regulatory and Data Restrictions

- The system needs to adhere to:
 - EU261/2004 specifications for compensation for flight delays.
 - GDPR for managing user data, such as wallet addresses, flight details, and account setup.
- Only hashed or anonymized references may be stored on-chain; personally identifiable information (PII) cannot.
- Industry-standard security (encryption in transit and at rest) must be enforced by off-chain storage (Supabase/PostgreSQL).

4. Limitations of the Technology Stack

- Frontend: Next.js (React framework) must be used for development.
- Backend/API: Supabase or a comparable PostgreSQL backend must be integrated.
- Blockchain Layer: Ethereum mainnet or, if gas costs need to be kept to a minimum, a supported Layer 2 (such as Polygon or Arbitrum).
- Wallet Integration: Ethereum provider API (ethers.js) with MetaMask.

- Oracle: For real-time or event-driven updates, a flight-status data source or a custom Oracle simulation are required.

5. Platform and Hardware Restrictions

- The system needs to facilitate:
 - MetaMask-compatible web browsers include Chrome, Firefox, Edge, and Brave.
 - Desktop responsive user interfaces.
- There is no presumption of specialized servers or dedicated hardware acceleration.
- Decentralized blockchain infrastructure is required for smart contract activities to run smoothly.

6. Limitations on Reliability and Fault Tolerance

- Blockchain network congestion or outages could cause delays in:
 - Creation of policies
 - Execution of payout
 - Oracle-driven modifications
- The blockchain record of truth takes precedence over centralized components (Supabase, hosting), which must maintain 99%+ uptime.
- False payouts or state transitions cannot be caused by Oracle failures.

7. Security Limitations

- Web3 security guidelines must be adhered to in all wallet interactions, including:
 - Transactions with signatures
 - Requests for permission
 - The web application does not handle private keys.
- Smart contracts need to be tested for:
 - Reentrancy
 - Overflows of integers
 - Unauthorized modifications to the state
- External oracle sources need to be cryptographically validated or trusted.

8. Constraints on Interoperability and Protocol

- The system must adhere to industry standards, such as:
 - JSON-RPC for Ethereum
 - Web3 provider standards for Ethers.js

- Frontend-backend communication via HTTPS
- Smart contracts need to adhere to accepted best practices from:
 - OpenZeppelin ERC-standards (if relevant)
- Supabase-supported SQL/REST communication standards must be followed by the backend.

3. Functional Requirements

3.1 - FR01: Flight Registration

Feature	Flight Registration
Description	Passengers must be able to register a specific flight that they intend to travel on.
Technical Issues	The system must prevent duplicate registrations for the same passenger and flight instance.
Dependencies	Relies on Supabase for user authentication and data storage.
Criticality	High

3.2 - FR02: Mock Oracle Service

Feature	Mock Oracle Service
Description	A mock oracle service must be able to send a POST request to the backend. This should contain the actual landing timestamp for a specific registered flight.
Technical Issues	Must validate that the flight exists and has been registered before submitting data to the contract. The time format must be consistent with what the smart contract expects
Dependencies	Depends on flight registration and on the smart contract to have the ability to store actual landing time.
Criticality	High

3.3 - FR03: Claim Eligibility Calculation

Feature	Claim Eligibility Calculation
Description	The smart contract must calculate whether the flight arrival delay is $\geq$ 3 hours. If eligible the claim status should move to Pending, otherwise it should go to Not Eligible.
Technical Issues	Time differences across different time zones must be accounted for.
Dependencies	Requires accurate input from the mock oracle and the smart contract storage structure to maintain claim state.
Criticality	High

3.4 - FR04: Claim Approval / Rejection

Feature	Claim Approval / Rejection
Description	Once a claim is marked as Pending, the airline must be able to respond, by approving or rejecting a claim with evidence, in a seven day dispute window.
Technical Issues	Role-based access control is required so only airline accounts can perform the action. Evidence must be stored off chain with only a hash stored on chain.
Dependencies	Dependent on claim eligibility calculation and Supabase storage for evidence and contract function to record dispute.
Criticality	High

3.5 - FR05: Claim Evidence Collection

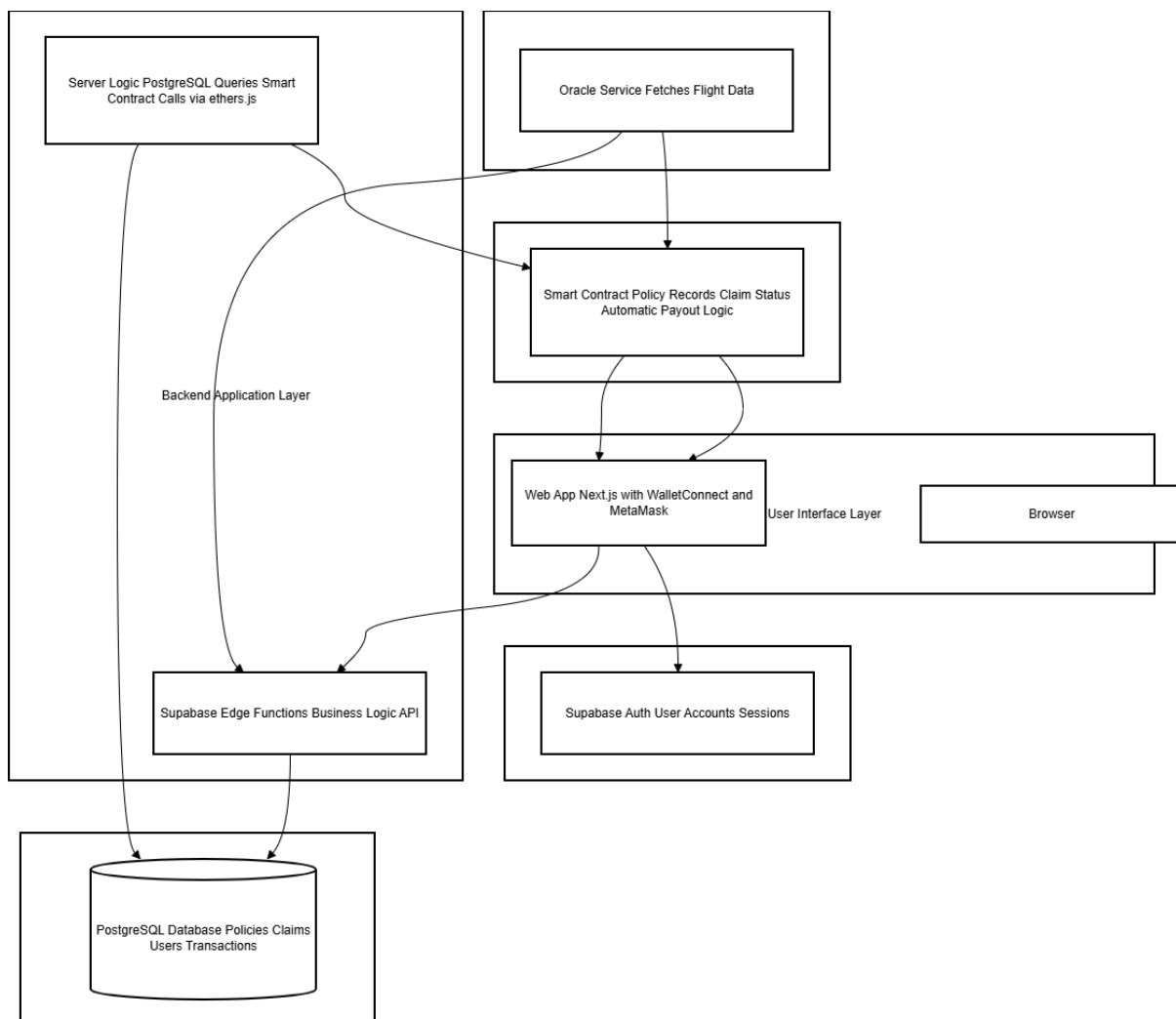
Feature	Claim Evidence Collection
Description	The passenger must be able to view the relevant evidence and collect a PDF dossier of the evidence.
Technical Issues	The PDF must clearly map blockchain proof
Dependencies	Requires Supabase for evidence retrieval and PDF generation logic.
Criticality	Medium

3.6 - FR06: Live Claim Status

Feature	Live Claim Status
Description	The system must display the real-time claim status updates (Registered → Pending → Approved / Rejected) to both passenger and airline users.
Technical Issues	Requires syncing on-chain events with off chain UI state.
Dependencies	Requires API helpers to direct blockchain event subscriptions.
Criticality	Medium

4. System Architecture

The system's service-oriented architecture facilitates eligibility verification, compensation processing, and claim submissions. The user-facing interface, the backend application services, and the external blockchain integrations make up the platform's three primary levels. The backend is responsible for executing smart contract functions (flight registration, delay submission, claim payouts). Travelers can register flights, and monitor their compensation status via the frontend module. Secure REST APIs are used to transfer all interactions to the backend. User authentication, claim validation, delay verification, compensation computations, and notification workflows are all handled by the backend service layer. Additionally this layer controls data durability by storing user information, travel details, and claim histories in a centralised database.



5. High-Level Design

5.1 System Architecture Overview

The system consists of four main layers:

1. **Client Layer (Passenger and Airline UI)**

Implemented using Next.js and React.

Users can interact with the system via standard web interfaces.

All communication with the backend happens through HTTP API calls.

2. **Backend Layer (Next.js API)**

Handles application logic flight registration, evidence uploading and communication between components.

The backend is also responsible for executing blockchain transactions using ethers.js using a server side key.

3. **On-Chain Layer (Smart contracts on Ethereum Sepolia)**

Contains the state machine that controls claim status transactions based on the flight arrival time and the Airlines response to the claim

Events sent by the contract are used to synchronise chain state back into the database.

4. **Off-Chain Data Layer (Supabase Postgres + Supabase Storage)**

- **Supabase Postgres** - Stores structured data such as users, claims, timestamps and state history.
- **Supabase Storage** - Stores unstructured files like airline evidence and the generated PDF dispute dossiers.

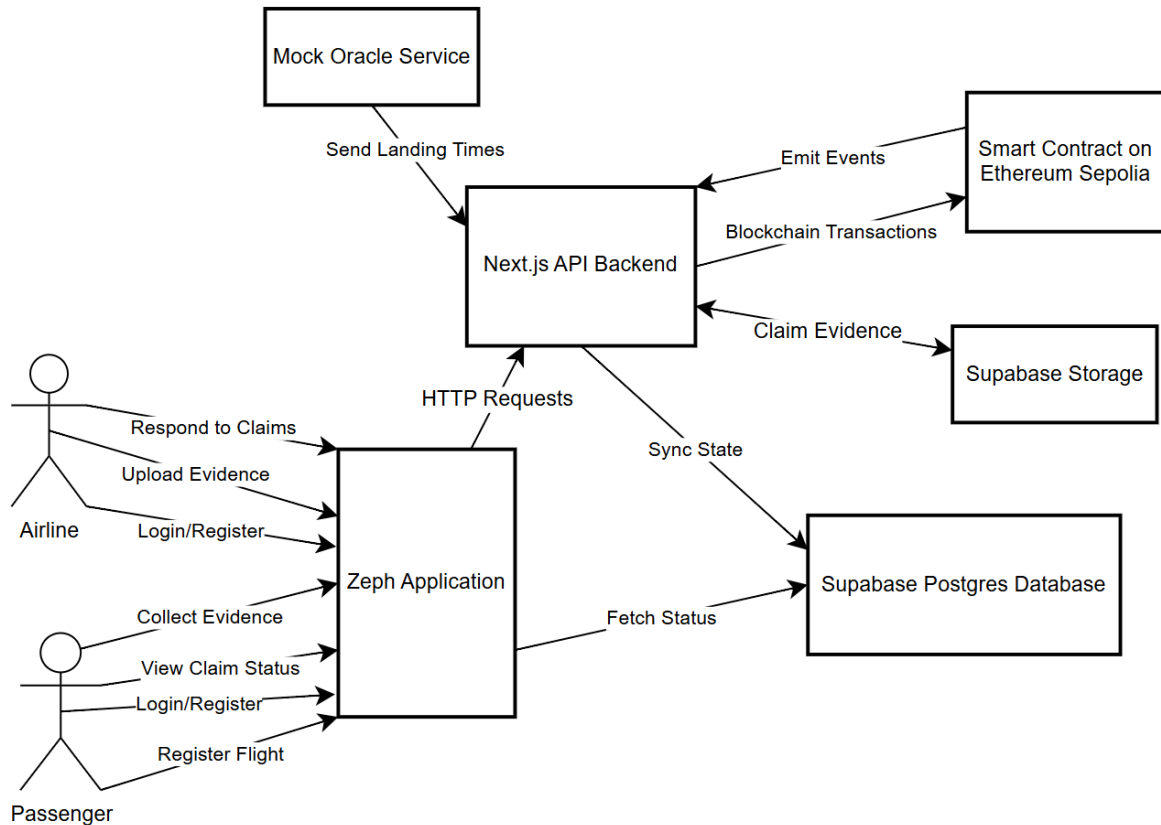


Figure 5.2: Context Diagram

This context diagram shows the Zeph application and its interactions with external system components that are involved in the process of automating EU261 claims.

- **Passenger:** Can; log in / register, register a flight, view claim status updates, and collect all available evidence relating to their claim.
- **Airline:** Can log in / register, respond to pending claims (approve / reject) within the seven day time window, and upload evidence.
- **Mock Oracle Service:** Simulates real-time flight arrival data by sending landing timestamps to the backend.
- **Next.js API Backend:** Serves as the core processing engine for the system. HTTP requests from application, forwards blockchain transactions via ethers.js, receives oracle data, interacts with Supabase DB, updates claim states.
- **Smart Contract on Ethereum Sepolia:** Implements the EU261 claim logic. Receives transactions from backend, evaluates delays, updates on-chain states, and emits events that notify the system of changes.
- **Supabase Storage:** Stores airline-uploaded evidence files.
- **Supabase Postgres Database:** Maintains all structured off-chain data; user profiles, registered flights, and claim states.

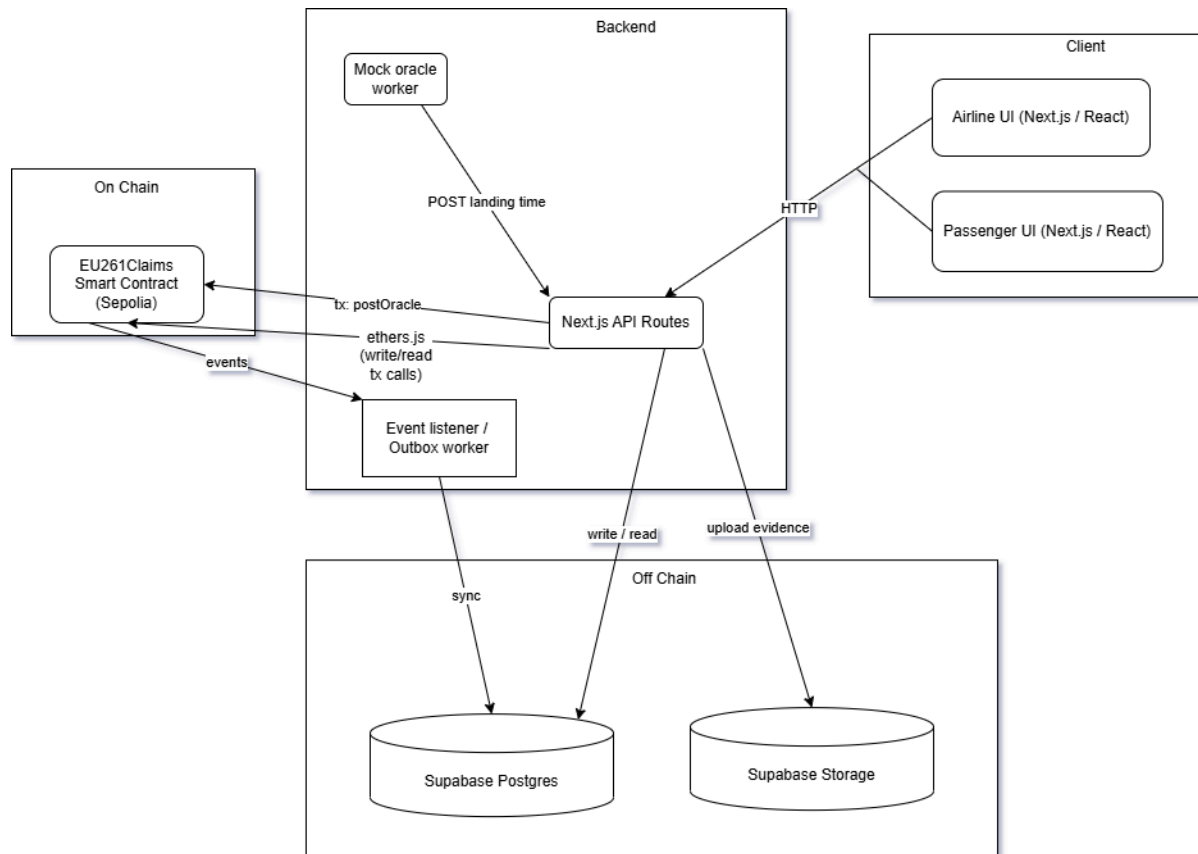


Figure 5.3 Component Diagram

This component diagram shows how Zeph's major system modules interact across the client, backend, on-chain and off-chain layers. Passenger and Airline UIs communicate with the backend using standard HTTP requests. The backend handles all blockchain interaction using ethers.js. This allows smart contract calls to be trusted and controlled. When the contract sends events the event listener will synchronise the new state into the postgres database. Evidence documents are stored separately in Supabase storage. This design keeps private keys away from the client and maintains data integrity across on-chain and off-chain components.

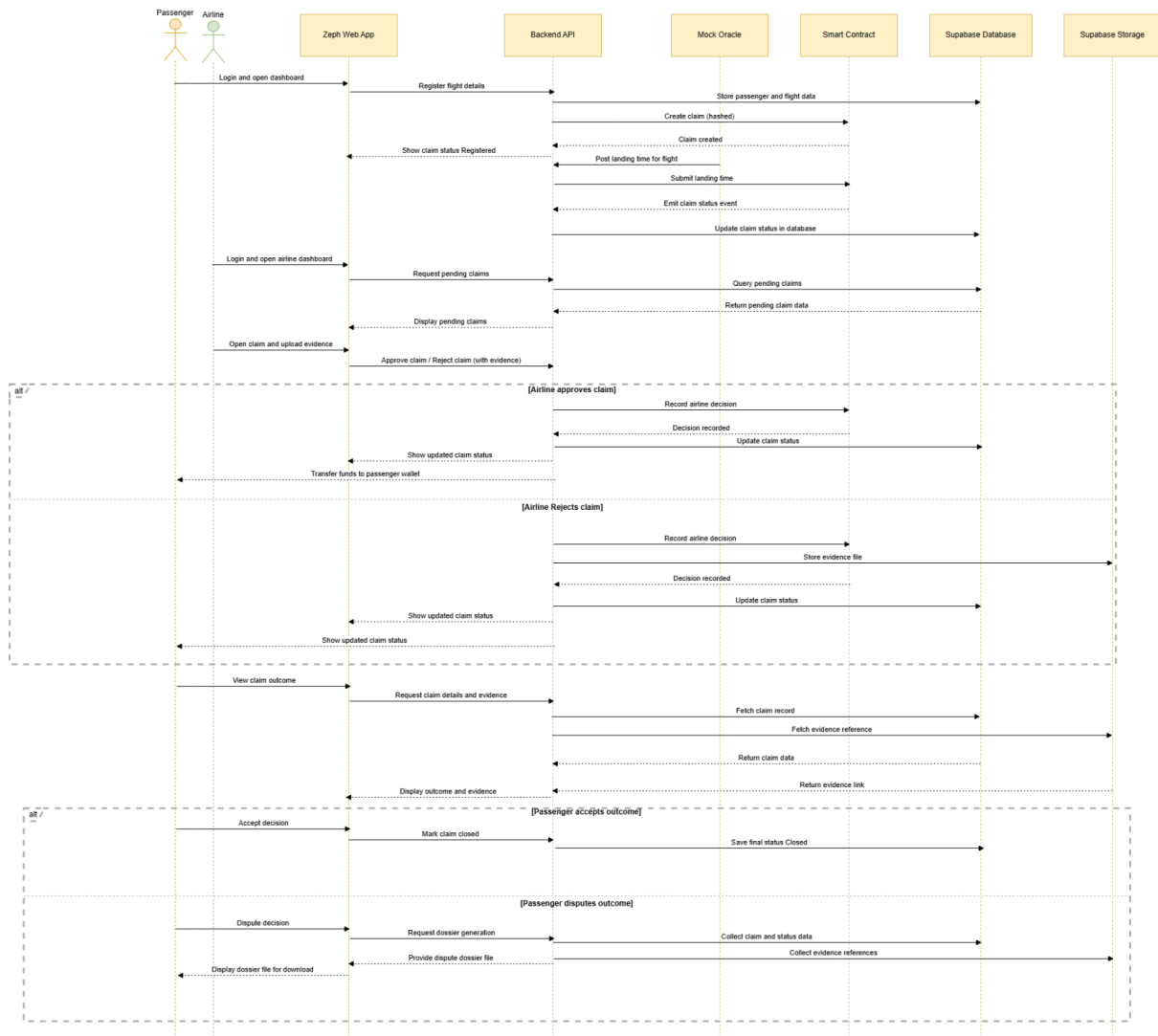


Figure 5.4 shows the Sequence Diagram

6. Preliminary Schedule

- Phase 1: Planning and Requirements
- Compile and complete the system requirements
- Describe high-level workflows and user journeys.
- Describe blockchain interactions, such as policy development, claim validation, and payouts.
- Create preliminary architectural diagrams
- Determine the main dependencies and hazards.
- Outputs: Initial system architecture and requirements document

Phase 2: Technical Configuration and System Architecture

- Complete the database architecture, blockchain, frontend, and backend.
- Set up environments for development
- Create workflows and a GitHub repository.
- Set up the PostgreSQL instance and Supabase project.

Dependencies: Phase 1

Phase 3: Front-end Development

- Construct the Next.js user interface
- Establish a wallet connection using MetaMask or WalletConnect.
- Make flight entries and claim filing forms.
- Create a dashboard to view payouts and policy status.

Dependencies: Phase 2

Phase 4: Database and Backend Development

- Put Supabase Auth into practice (user accounts + sessions)
- Make database schemas for transactions, policies, users, and claims.
- Construct Edge Functions for Backend Logic
- Connect the frontend to API calls
- Dependencies: Phase 3 can operate concurrently, but a robust database schema is required.

Dependencies: Phase 3 can operate concurrently, but the database schema needs to be reliable.

Phase 5: Development of Smart Contracts

- Implement policy formation, claim eligibility checks, payout logic
- Create and execute unit tests
- Deploy to the Ethereum test network
- Connect backend services to Ethers.js

Dependencies: Architecture in Phase 2, with some reliance on Phase 4

Phase 6: Development and Integration of Oracle

- Use a flight-status oracle, either actual or simulated.
- Link Oracle to the smart contract
- Make logic for delay verification.
- Test the entire data flow

Dependencies: Phase 5

Phase 7: Testing and System Integration

- Combine Oracle, databases, smart contracts, frontend, and backend.
- Implement failure, edge case, and functional testing.
- Verify payout automation
- Make UI/UX improvements
- Dependencies: Every stage of development

Dependencies: All development phases

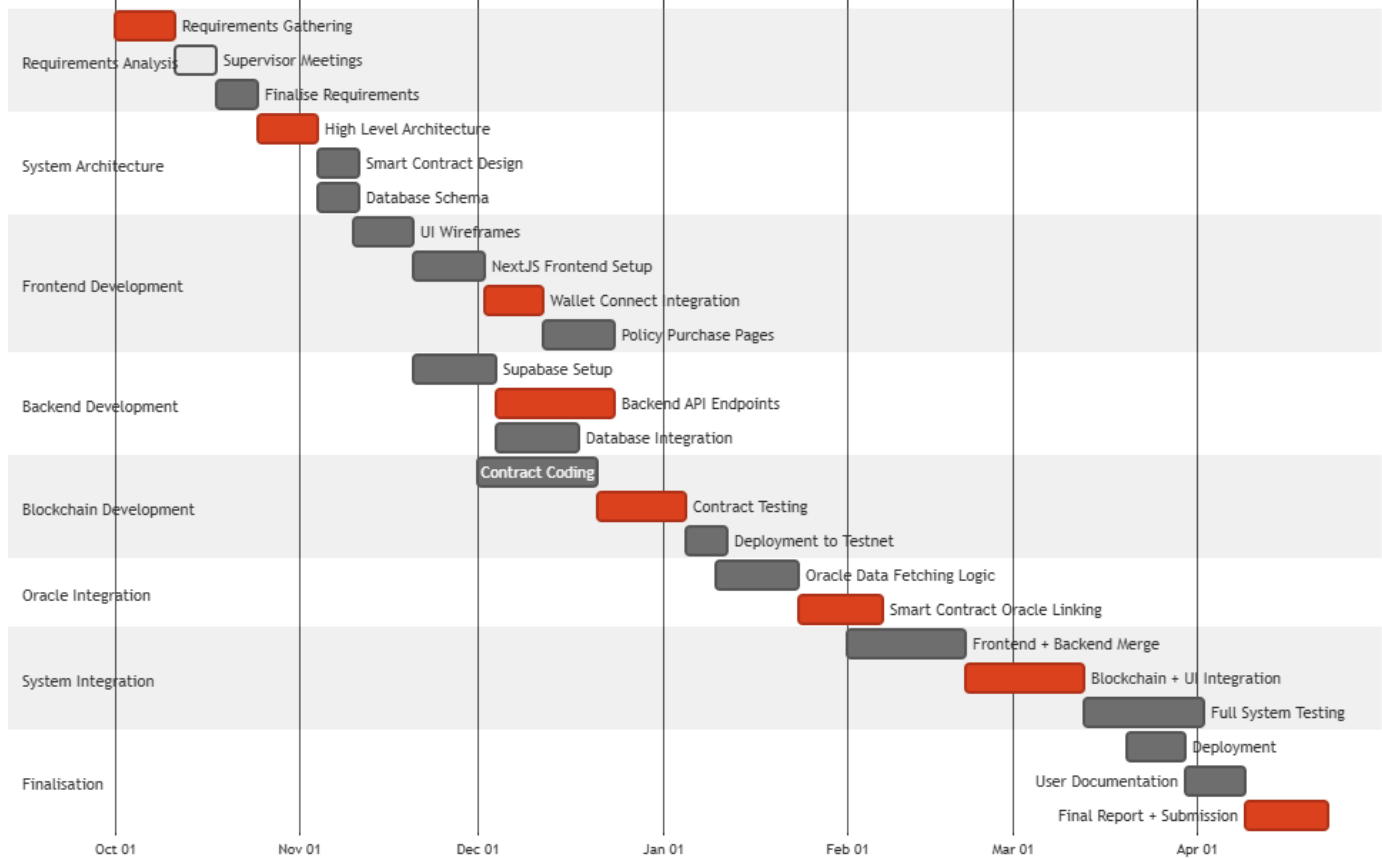
Phase 8: Complete Deployment and Record-Keeping

- Deploy the front end and back end.
- Release the final version of the contract
- Write technical and user documentation.
- Get the presentation materials ready.

Dependencies: Phase 7

6.1 Gantt chart

Detailed Project Gantt Chart (Oct 2024 - Apr 2025)



7. Appendices

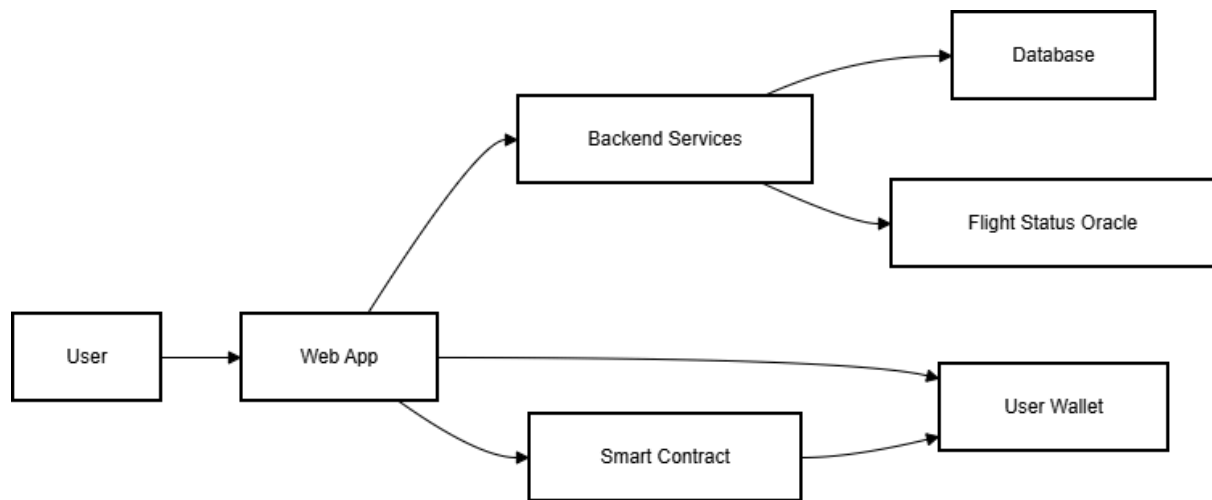


Figure 7.1 Shows a Simple Overview of the System

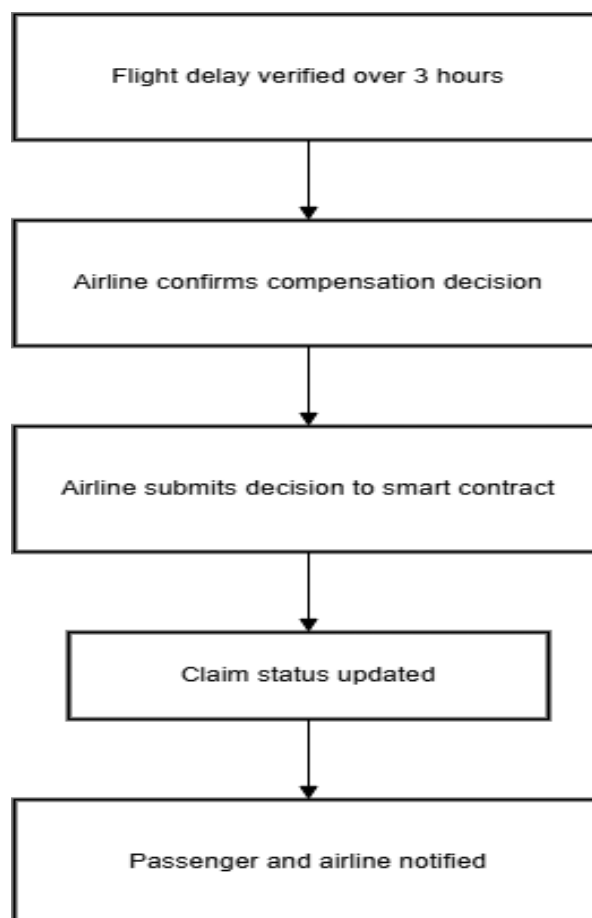


Figure 7.2 Shows the Smart Contract Flow

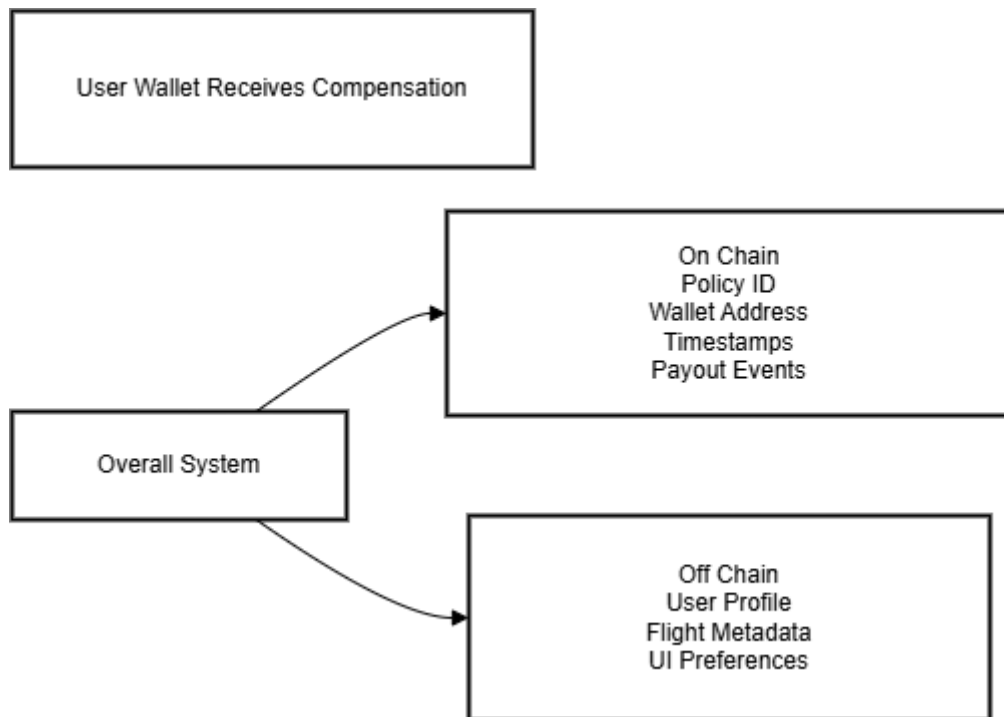


Figure 7.3 Shows the Data Storage breakdown

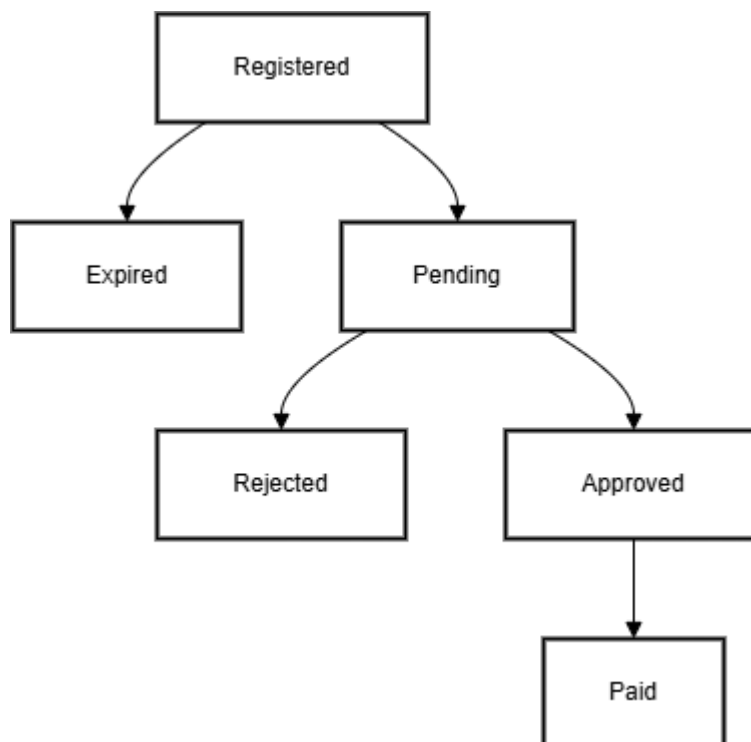


Figure 7.4 Shows the Claim / Compensation Lifecycle